

Bases de Datos 2

Teoría 09

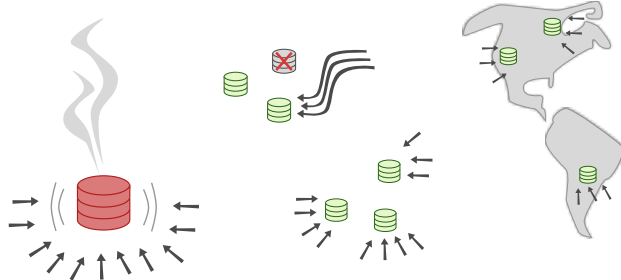
Julián Grigera

Consistencia y Replicación

En NoSQL

Replicación

Performance de Lectura



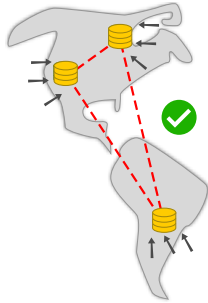
Hay 2 motivos primarios para replicar datos:

- 1) **Fiabilidad**: si un nodo se cae, se puede desviar el tráfico a otra copia
- 2) **Performance** ya sea por
 - Volumen de **tráfico**
 - Latencia por **distancia**

En el caso del escalado **geográfico**, mantener las copias **consistentes** puede incurrir en uso de **ancho de banda** y volver a afectar la latencia

Replicación

Y Performance



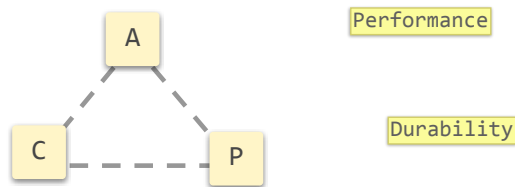
Los problemas de escalabilidad suelen manifestarse en forma de problemas de rendimiento. Colocar copias de los datos cerca de los procesos que los utilizan puede reducir el tiempo de acceso.

Se puede buscar consistencia “global”, pero

- Sería muy **lento** coordinar las réplicas
- El método de caché para **acelerar** genera **más latencia**
- “La cura es peor que la enfermedad”

Compromiso

Performance / Consistencia



Normalmente, un proceso que realiza una operación de lectura sobre un dato, espera que la operación devuelva un valor que muestre los resultados de la última operación de escritura sobre ese dato.

En MongoDB

Ejemplo de Quorums y estrategias

Tipo	Descripción	Usos
local (default)	Lee lo que tenga el nodo local, sin esperar replicación ni consenso.	Lecturas rápidas donde toleras ver datos ligeramente viejos o no confirmados. Logs, métricas, dashboards internos.
available (sharding)	Devuelve lo que esté disponible, incluso si falta un shard o hay inconsistencia entre shards.	E-commerce global, catálogos grandes, vistas “best-effort” donde es mejor mostrar algo que fallar.
majority	Solo ve datos que alcanzaron a la mayoría de los nodos.	Reservas, stock, estados de cuenta, cualquier cosa donde leer datos viejos rompa la lógica de negocio.
snapshot	Foto consistente de los datos dentro de una transacción.	Reportes, análisis, operaciones largas que necesitan una vista estable de los datos.

En MongoDB

Ejemplo de Quorums y estrategias

writeConcern	Qué significa	Cuándo usarlo
{ w: 0 }	El cliente no espera confirmación de nada (ni primario).	Perder datos no importa (inusual).
{ w: 1 }	Se considera exitosa cuando la recibe el primario (no espera réplica).	Alta escritura, se tolera perder las últimas operaciones si el primario cae.
{ w: N }	Espera confirmación de N nodos (incluido el primario).	Casos muy específicos.
{ w: "majority" }	Espera confirmación de la mayoría de los nodos del replicaset.	Casos críticos: reservas, dinero, estados.
{ w: 1, j: true } { w: "majority", j: true }	Además, asegura que la operación se haya escrito en el journal del nodo.	Cuando la durabilidad frente a crash de máquina es importante.

En MongoDB

Ejemplo de Quorums y estrategias

```
db.collection("users").withWriteConcern({
  w: "majority",
  wtimeout: 5000
});
```

```
users.findOne(
  { name: "Ana" },
  { readConcern: { level: "local" } }
);
```

Lectura rápida, sin esperar consenso

Escritura rápida, solo primario

```
users.withWriteConcern({ w: 1 }).insertOne({
  name: "EventoLog",
  ts: new Date()
});
```

Familia de Columnas

Cassandra y otras



BBDD NoSQL y Columnas

Conceptos

Column-Oriented **COLUMN
STORAGE**
Wide Column
Column Family *Columnar*

Existen varios términos alrededor de las BBDD NoSQL orientadas a columnas. Es todo bastante confuso.

Basta con entender que solo hay dos aspectos importantes:

- El modelo de datos
- El almacenamiento

No son independientes, pero sirve separarlos para entender.

BBDD NoSQL y Columnas

Row Storage vs. Column Storage

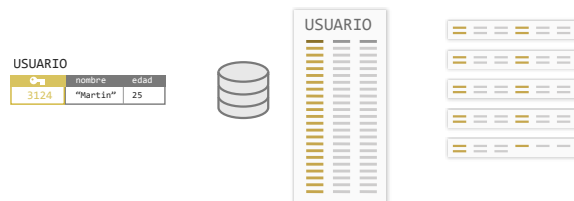
rowid	id	name	age
9987	3124	"Martin"	25
9988	3127	"Ana"	32
9989	3128	"Pablo"	29
9990	3130	"Santiago"	45
9991	3131	"Lorena"	46
9992	3132	"Mariano"	22
9993	3133	"Silvia"	56
9994	3134	"Juana"	21
9995	3135	"Claudio"	76

Veamos un ejemplo para entender los tipos de almacenamiento

Existen dos: orientado a filas / orientado a columnas.

Row Storage

Almacenamiento en filas



En las BBDD tradicionales, incluso en muchas NoSQL, se almacena por fila.

Si buscamos varias celdas de una fila, a nivel de disco que se organiza en bloques tenemos un caso de uso óptimo.

Si buscamos una sola celda/columna, a nivel de disco obtenemos igualmente todas - no tan óptimo.

Hay optimizaciones, pero no son parte de la lógica esencial del almacenamiento por fila.

BBDD NoSQL y Columnas

Almacenamiento en filas - Row Storage

```
9987 3124 "Martín" 25 9988 3127 "Ana" 32 9989 3128  
"Pablo" 29
```

```
9990 3130 "Santiago" 45 9991 3131 "Lorena" 46 9992  
3132 "Mariano" 22
```

```
9993 3133 "Silvia" 56 9994 3134 "Juana" 21 9995 3135  
"Claudio" 76
```

Físicamente, las filas se guardarán en bloques de tamaño fijo.

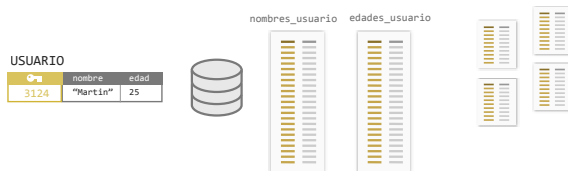
Dependiendo del tipo de consulta puedo estar haciendo mucho I/O, según el nro de bloques, por ej:

SELECT * FROM users -> nro de llamadas I/O será más óptimo, ya que necesitamos toda la info tal cual está agrupada en disco

SELECT AVG(age) FROM users -> más I/O con información superflua, no necesitamos toda la info que leemos

BBDD NoSQL y Columnas

Almacenamiento en columnas - Column Storage



En las BBDD "column-store" se almacena **físicamente** por columna.

BBDD NoSQL y Columnas

Almacenamiento en columnas - Column Storage

```
3124:9987 3127:9988 3128:9989 3130:9990 3131:9991  
3132:9992 3133:9993 3134:9994 3135:9995
```

```
"Martín":9987 "Ana":9988 "Pablo":9989 "Santiago":9990 "Lorena":9991
```

```
"Mariano":9992 "Silvia":9993 "Juana":9994 "Claudio":9995
```

```
25:9987 32:9988 29:9989 45:9990 46:9991 22:9992 56:9993 21:9994 76:9995
```

Físicamente, se guardan las columnas, junto a su **rowid**

Todo cambia, ahora con estos ejemplos:

SELECT * FROM users -> nro de llamadas I/O será ineficiente, necesitamos recorrer muchos bloques para reconstruir una fila.

SELECT AVG(age) FROM users -> muy eficiente, solo recorreremos el bloque/s de la edad.

Permite compresión, dado que hay un solo dato en el almacenamiento y está ordenado.

BBDD de Familias de Columnas

Apache Cassandra



```
SET Customer['mfowler']['city']='Boston';  
GET Customer['mfowler']['web'];
```

Cassandra es un producto popular de Familias de Columnas.

Diseño: promueve “Query First Approach”

Arquitectura de distribución: utiliza P2P

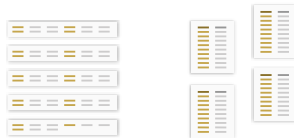
Hablemos de normalización... ejemplo de blog + comentarios

Cassandra

Column-Stored o Row-Stored?



Cassandra



```
"Employees" : {  
  row1 : { "ID":1, "Last":"Cooper", "First":"James", "Age":32},  
  row2 : { "ID":2, "Last":"Bell", "First":"Lisa", "Age":57},  
  row3 : { "ID":3, "Last":"Young", "First":"Jospeh", "Age":45},  
  ...  
}
```

Físicamente, cómo almacena? Filas o columnas?

Ninguna de las 2

Almacenamiento por filas, pero particionadas

Está pensada para la distribución.

Apache Cassandra

Query-First Approach



En Cassandra, el modelado de datos es “query-driven”.

Los datos se modelan en torno a consultas específicas.

Las consultas se diseñan para acceder a una única tabla:

- las entidades implicadas deben estar en la misma tabla
- el acceso a los datos (lecturas) será muy rápido.

Esto implica que una entidad puede estar replicada en muchas tablas

Apache Cassandra

Query-First Approach

id	name	age	lang
3124	"Martín"	25	2231
3127	"Ana"	32	2231
3128	"Pablo"	29	2231
3130	"Santiago"	45	2231
3131	"Sara"	46	2232
3132	"José"	22	2232
3133	"Franz"	56	2234
3134	"Verena"	21	2234
3135	"Helmut"	60	2234

id	name	key
2231	"Español"	sp
2232	"Portugues"	pt
2233	"English"	en
2234	"Deutsch"	de

Ejemplo partiendo de un modelo típico de RDBMS

Apache Cassandra

Query-First Approach

users_by_lang		
lang	name	age
es	"Martín"	25
es	"Ana"	32
es	"Pablo"	29
es	"Santiago"	45
pt	"Sara"	46
pt	"José"	22
de	"Franz"	56
de	"Verena"	21
de	"Helmut"	60

Partiendo de un caso de uso, modelamos la tabla detrás de la query necesaria.

- La clave primaria (pk) puede ser cualquier combinación de atributos.
- También hay una **partition key** que es parte de la **pk**.
- La partition key indica en qué nodo/s físico se guarda cada tupla.
- En este caso, "lang" pasa a ser la partition key (el resto de la pk son **clustering keys** usadas para ordenar).

¿Qué significa esto para las queries? Hay que incluir la **partition key** para no recorrer todos los nodos.



La no posibilidad de usar JOIN determina el modelado.

Toda consulta se diseña para buscar en una sola tabla.

En consecuencia, las consultas son además más simples, no solo más veloces.

Apache Cassandra
Query-First Approach

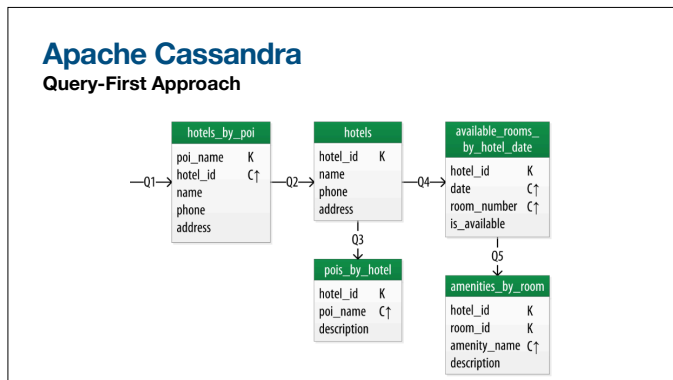
available_hotels_by_city

- city_name (Primary Key)
- date
- hotel_id
- hotel_name
- is_available
- from_price

hotels

- hotel_id (Primary Key)
- hotel_name
- phone
- address

Otro caso de uso y ejemplo de denormalización / enfoque query-first



Ejemplo de diseño por flujo de queries

https://cassandra.apache.org/doc/stable/cassandra/data_modeling/data_modeling_logical.html

Apache Cassandra
Creación de Tablas

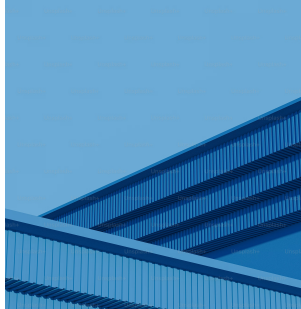
```
CREATE TABLE available_hotels_by_city (
  city_name    VARCHAR,
  date         DATE,
  hotel_id     UUID,
  hotel_name   VARCHAR,
  is_available BOOLEAN,
  price        FLOAT,
  PRIMARY KEY ((city_name), date, hotel_id)
);
```

Ejemplo de definición, para ver claramente la definición de claves

Entre () se marca la *partition key*, que siempre es parte de la *primary key*

Apache Cassandra

Arquitectura



Cassandra

Keyspaces, Tablas, etc



En Cassandra, un **keyspace** es una BD en RDBMS

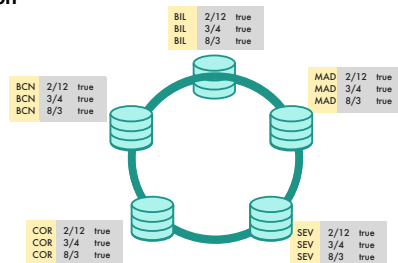
Las tablas tienen particiones, necesariamente.

Prácticas asociadas a este hecho:

- Siempre se debe guardar junto lo que se recupera junto
- Evitar particiones sin límite de crecimiento (*unbounded*)
- Evitar particiones que concentren los reads en un momento dado

Cassandra

Distribución



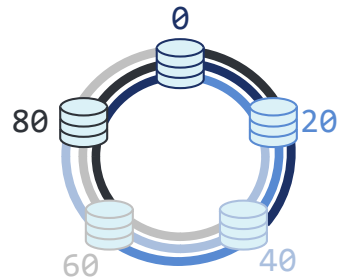
Las claves determinan el nodo en el que se guardan los datos.

Pero qué pasa si pierdo un nodo?

Necesito replicación

Cassandra

Particiones y Distribución



La *partition key* determina el nodo en el cual se guarda la fila.

Cada nodo tiene entonces asignado parte del espacio de las claves.

Además, hay replicación entre nodos (aunque tengan espacios diferentes)

Cada nodo se asigna un nro de **tokens**.

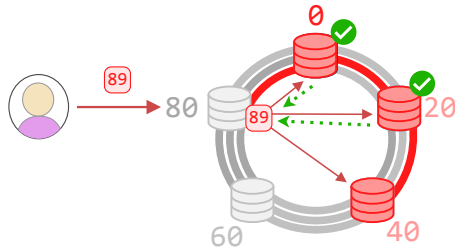
En realidad, las partition keys pasan por una función de hash que da un **token**

Es P2P - cualquier nodo puede hacer todo.

Un nodo atiende la lectura, y luego se ocupa de delegar al que tenga el dato.

Cassandra

Quora



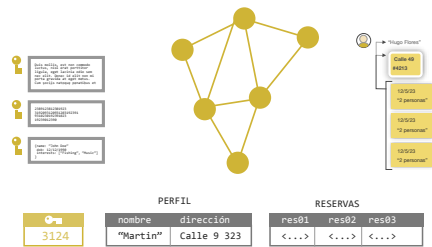
Cassandra

Casos de uso

- Patrones de lectura **predecibles**
- Logging
 - Actividad Web
 - Salud
 - IoT
- Mucha demanda de **escritura**

Final

Repaso y trivía



Casos de Uso

Trivia!

- Datos muy Relacionados
- Sesiones de Usuario
- Analítica
- Aplicaciones de E-Commerce
- Perfil de Usuario
- CMS
- Carrito de Compras
- Datos con Expiración
- Logs

¿Qué BD uso?

Criterios para elegir tecnología

Esta slide está intencionalmente en blanco
